

A Hybrid Retrieval and Cross-Encoder Reranking Architecture for AI-Assisted Resume Screening and Talent Acquisition

Daniel Leve Manickam D A

230701060@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Darshan M

230701061@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Dayanithi V

230701064@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Deepa S

230701065@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Dr. J. Jinu Sophia

jinusophia.j@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Abstract—Large applicant collections require candidate generation to be efficient without reducing screening to exact keyword overlap. This paper describes an implemented retrieval-and-reranking pipeline for PDF resumes. An Amazon S3 event invokes asynchronous Amazon Textract processing, and the resulting text and layout blocks are retained as provenance-bearing JSON. A deterministic section-aware procedure divides each resume into candidate chunks. Voyage voyage-4-large produces 1024-dimensional representations, while MongoDB Atlas independently performs BM25 lexical retrieval and vector semantic retrieval. Reciprocal Rank Fusion combines the two chunk rankings and deduplicates them into a candidate pool. Candidate documents are reconstructed from stored chunks and jointly evaluated with the job description by the configured Voyage rerank-2.5 cross-encoder. Its output is treated as a semantic relevance score rather than a hiring probability. A repository-backed pilot illustrates complementary retrieval coverage and rank movement, but does not establish ranking accuracy. Requirement-to-resume evidence alignment, explainable recommendations, alternative-opening matching, and candidate communication are presented only as proposed extensions for human-reviewed decision support.

Keywords—hybrid retrieval, BM25 lexical retrieval, vector semantic retrieval, Reciprocal Rank Fusion, candidate chunk, cross-encoder reranking.

I. INTRODUCTION

Recruitment screening is an information-retrieval problem before it is a decision problem. A technical vacancy may attract resumes that describe similar competence with different vocabulary, structure, and detail. Exact-term filtering can miss relevant applicants, while applying an expensive joint language model to every job–resume pair does not scale gracefully. The practical requirement is therefore to retrieve broadly, rerank selectively, and retain enough provenance for human review.

Bi-encoders make corpus search efficient because documents can be embedded before a query arrives [1], [2]. Their independent representations, however, do not model every interaction between a job description and a complete resume. Cross-encoders provide richer pairwise comparison but require

evaluation of each selected pair [3]. The implemented system joins these approaches with lexical retrieval: MongoDB Atlas Search supplies BM25 rankings, Atlas Vector Search supplies semantic rankings, and Reciprocal Rank Fusion (RRF) merges them before cross-encoder reranking [4].

This work contributes: 1) an event-driven, PDF-only extraction path that preserves Textract page and block data; 2) deterministic, section-aware candidate chunks with idempotent indexing; 3) chunk-level BM25 and vector retrieval with implementation-aligned candidate aggregation; 4) reconstruction and rerank-2.5 scoring that preserves retrieval provenance; and 5) an evaluation framework that separates observed pilot behaviour from unmeasured quality. Evidence alignment and communication are future extensions rather than implemented claims.

II. BACKGROUND AND RELATED WORK

A. Resume Document Intelligence

Resume processing begins with document recovery rather than semantic judgement. Layout, reading order, repeated headings, and page boundaries affect later retrieval. Amazon Textract supports asynchronous text detection for documents stored in S3 and reports completion through Amazon SNS [5]. The current system preserves the returned text, blocks, page numbers, and geometry; it does not claim that Textract directly produces verified skills, education, or experience entities. Research on resume parsing and skill-span extraction motivates later structured analysis, but those capabilities must be evaluated separately [6], [7].

B. Lexical and Dense Retrieval

BM25 rewards query terms according to within-document frequency, document length, and corpus rarity [8]. It is useful when a job description contains exact technologies, certifications, or role names. Dense retrieval maps differently worded but related passages into a shared vector space [1], [2]. Neither signal is universally sufficient: lexical search can miss paraphrases, whereas dense search can underweight exact constraints.

C. Hybrid Retrieval and RRF

Hybrid retrieval retains both signals without forcing raw scores onto a common scale. RRF combines ordinal positions

[4]. Here, the ranked objects are resume chunks: all retrieved chunks contribute to their candidate, and the fused result is then deduplicated. This differs from fusing one pre-aggregated score per resume.

D. Cross-Encoder Reranking

BERT reranking demonstrates the value of jointly encoding a query and a candidate passage [3]. The deployed reranker similarly receives a job description and an array of reconstructed candidate documents. Its score orders semantic relevance within the retrieved pool; it is not calibrated as a probability of hiring success and is not combined numerically with the RRF score.

E. Explainability, Fairness and Human Oversight

Post-hoc explanation methods can indicate influential features [9], [10], but employment screening needs verifiable source evidence, not merely fluent rationales. Removing explicit identity fields also does not remove institutional, geographic, or career-history proxies [11]. Accordingly, proposed explanations will be constrained by stored resume evidence, and recruiters will remain responsible for requirements, corrections, and final decisions.

III. PROBLEM FORMULATION AND MATHEMATICAL MODEL

A. Candidate Chunk Representation

Let candidate c_i contain an ordered set of chunks $D_i = \{x_{i1}, \dots, x_{in_i}\}$. The implementation embeds each chunk and the job description in $\mathbb{R}^d$, with $d = 1024$ for the configured voyage-4-large model [12], [25]. Document chunks use the model's document input mode and the job description uses query mode.

B. Vector Similarity

For job-description embedding q and chunk embedding x_{ij} , vector retrieval is represented by cosine similarity:

$$s_{\text{vec}}(q, x_{ij}) = \frac{q^\top x_{ij}}{\|q\|_2 \|x_{ij}\|_2}. \quad (1)$$

MongoDB Atlas Vector Search executes the approximate-nearest-neighbour query over stored chunk vectors [13].

C. BM25 Retrieval

For query terms $t \in q$ and candidate chunk x , the standard BM25 form is

$$s_{\text{BM25}}(q, x) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, x)(k_1 + 1)}{f(t, x) + k_1(1 - b + b|x|/\text{avgdl})}. \quad (2)$$

MongoDB Atlas Search computes the deployed score internally; the application consumes only the returned chunk order and does not manually normalise BM25 values [14].

D. Reciprocal Rank Fusion

Let L_m be the ranked chunks from method $m \in \{\text{lex}, \text{vec}\}$. The exact candidate-level behavior in fusion.py is

$$S_{\text{RRF}}(c_i) = \sum_m \sum_{\substack{x \in L_m \\ \text{cand}(x)=c_i}} \frac{1}{60 + r_m(x)}. \quad (3)$$

Thus, every retrieved chunk contributes; method weights are one. The best per-method rank retained for provenance is

$$r_m^*(c_i) = \min_{x \in L_m: \text{cand}(x)=c_i} r_m(x), \quad (4)$$

and remains null when that method did not retrieve the candidate. Sorting S_{RRF} and truncating after candidate deduplication yields the pool $\mathcal{C}_K$.

E. Cross-Encoder Reranking

All MongoDB chunks for c_i are ordered by chunk_index and reconstructed as D_i^* . The configured model returns

$$s_{\text{rank}}(c_i | q) = R(q, D_i^*), \quad c_i \in \mathcal{C}_K, \quad (5)$$

where R is rerank-2.5 [15], [26]. The final order is descending in this semantic relevance score. Rank movement is recorded as $\Delta r_i = r_i^{(2)} - r_i^{(3)}$; positive values indicate movement toward rank one.

F. Computational Workload

If N is the corpus size, K the retrieved pool, and c_x the conceptual cost of one query–document evaluation,

$$W_{\text{full}} = Nc_x, \quad W_{\text{two}} = Kc_x, \quad \frac{W_{\text{two}}}{W_{\text{full}}} = \frac{K}{N}. \quad (6)$$

This is an analytical pair-workload model, not measured latency or API-call count. The implementation submits the candidate-document array in one rerank request.

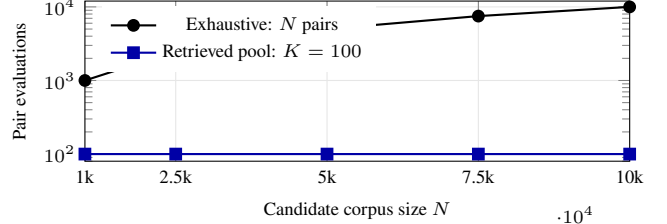


Figure 1. Analytical cross-encoder pair workload as the candidate corpus grows, with a fixed retrieved pool of $K = 100$.

G. Storage Model

For N candidates, average c chunks per candidate, $M = Nc$ total chunks, dimension d , b bytes per component, and metadata footprint m per chunk,

$$B_{\text{index}} \approx M(db + m) = Nc(db + m). \quad (7)$$

This analytical approximation excludes MongoDB index structures, replication, source JSON, and billing overhead.

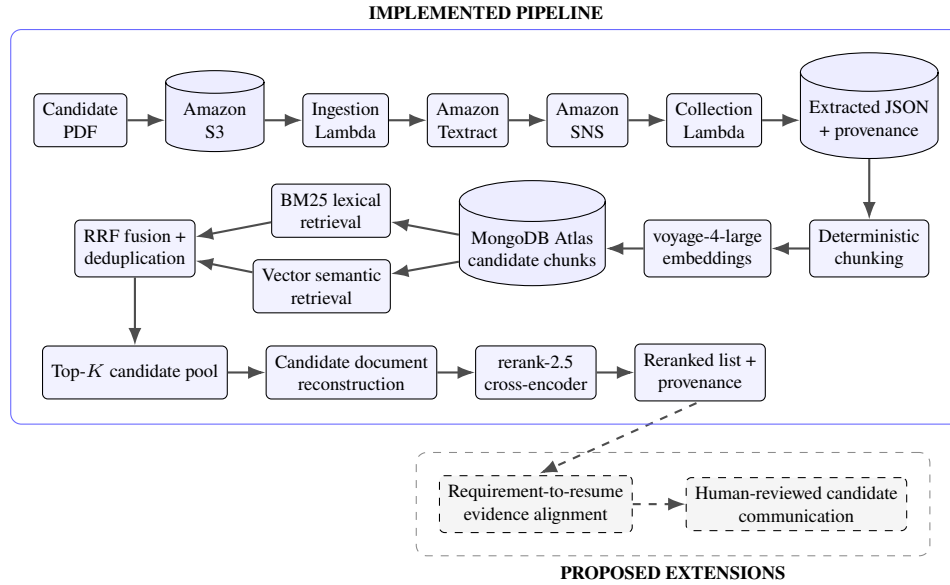


Figure 2. Hybrid retrieval and cross-encoder reranking pipeline with proposed evidence and communication extensions.

IV. SYSTEM ARCHITECTURE

A. Overall Architecture

Fig. 2 separates executable components from future extensions. The implemented path ends with a provenance-bearing reranked list. It contains no orchestrator, autonomous agents, Cognito, API Gateway, VPC, relational database, or DOCX path. The architecture is a retrieval decision-support pipeline rather than an autonomous hiring system.

B. PDF Ingestion and Textract Extraction

The S3 bucket accepts events only for PDF keys under incoming/. An object-created event invokes a Lambda function that calls StartDocumentTextDetection and supplies an SNS completion channel. A second Lambda consumes the completion message, follows every NextToken returned by GetDocumentTextDetection, and writes extracted/<job-id>.json. The record contains concatenated line text, the complete Textract block array, page count, original S3 location, processing timestamp, and pipeline version. Failed jobs are written separately.

C. Deterministic Chunking and Indexing

The chunker recognizes canonical headings for summary, skills, experience, projects, education, certifications, achievements, publications, languages, volunteering, and interests. It retains a header chunk and splits oversized sections at line boundaries with a configured 1200-character limit. If no headings are detected, it uses 1000-character windows with 100-character overlap. Chunk identifiers, section labels, ordinal positions, source keys, and page ranges are deterministic. A SHA-256 text hash makes unchanged re-indexing a no-op; changed content replaces stale candidate chunks.

Each chunk is embedded using input_type=document; the job description uses input_type=query. MongoDB stores the text, vector, candidate identifier, content hash, section, page range, and embedding configuration. Operational indexes

support candidate lookup and idempotency, while Atlas Search and Vector Search indexes support retrieval.

D. Hybrid Candidate Retrieval

The lexical pipeline applies the Atlas \$search text operator to chunk text and retains the returned BM25 order. The vector pipeline applies \$vectorSearch to the 1024-dimensional embedding field, using ten times the requested limit as the approximate-search candidate count, capped at 10,000. Both branches currently request 15 chunk hits by default. RRF then accumulates rank contributions, preserves matched chunk identifiers and section labels, deduplicates by candidate identifier, and returns up to 15 candidates.

E. Candidate Reconstruction and Reranking

For each retrieved candidate, all MongoDB chunks are fetched and ordered by chunk_index. Section labels are inserted into a reconstructed profile, which is conservatively limited to 30,000 characters. The implementation passes the job description and valid candidate documents to one Voyage rerank request. rerank-2.5 is the selected model for the current implementation; it is not described as the latest model. The output records rerank rank, semantic relevance score, rank change, reconstructed chunk identifiers, original source key, and all retrieval provenance. RRF and reranker scores remain separate because their scales and meanings differ.

F. Proposed Evidence and Communication Extensions

A future evidence layer will align each confirmed job requirement with supporting resume spans and their page/block provenance. It will distinguish supported, partial, absent, and review-required states, and prevent an explanation generator from introducing claims without stored evidence. Alternative-opening matching and candidate communication will consume this reviewed record. They are design directions, not current repository capabilities.

Table 1. Implemented component contracts and retained outputs

Operation	Main components	Retained output
Extraction	S3, Lambda, Textract, SNS	Text, blocks, pages, geometry, source key
Indexing	Chunker, Voyage, MongoDB	Chunk ID, section, page range, hash, embedding
Retrieval	BM25, vector search, RRF	Method ranks, fused score, matched chunks
Reranking	Reconstruction, rerank-2.5	Relevance score, rank change, provenance

V. IMPLEMENTATION

A. AWS S3 and Textract

AWS CloudFormation declares the infrastructure. The retained S3 bucket blocks public access and applies AES-256 encryption. Separate IAM roles restrict PDF reads, Textract initiation, SNS publication, result collection, and extracted or failed JSON writes.

B. Voyage and MongoDB Integration

The embedding client batches texts, validates query/document modes, retries transient failures, and avoids logging resume text. The indexer derives a stable candidate identifier from the PDF filename and stores chunk-level embeddings. Table 1 summarises the data retained across the pipeline.

C. Reranking Output Contract

The reranking worker reads a job description and its retrieval JSON, reports invalid reconstructions, sorts valid results by relevance score, and writes the reranked output. Its companion summary records input/output counts, failures, processing time, and score distribution.

Repository tests cover deterministic chunking, page propagation, content hashes, multi-chunk RRF, missing-rank preservation, top- K truncation, and reconstruction. The uploaded cache lists 68 collected cases with no cached failures.

VI. EVALUATION METHODOLOGY

A. Dataset and Experimental Setup

The available artefacts contain one backend-engineer job description and an executed retrieval/reranking output. This pilot is suitable for checking data flow, provenance, and rank transformations, but not for estimating general performance. A formal dataset should include consented, de-identified PDF resumes, multiple job families, and independent recruiter relevance labels. Train, tuning, and test partitions must prevent the same candidate or near-duplicate resume from crossing partitions.

B. Extraction and Retrieval Evaluation

Extraction precision, recall, and F1 will be calculated against page-level text and structure annotations. For a recruiter-labelled relevant set $\mathcal{R}_q$, retrieval coverage is

$$\text{Recall@}K(q) = \frac{|\mathcal{C}_K(q) \cap \mathcal{R}_q|}{|\mathcal{R}_q|}. \quad (8)$$

BM25/vector overlap and hybrid union measure complementarity, while MRR and nDCG@ K require graded or ordered relevance labels. The present pilot reports only set counts because no golden labels were found.

C. Reranking and Explanation Evaluation

With recruiter orderings, reranking will be evaluated using nDCG@ K , Kendall’s τ , Spearman correlation, and top- K agreement. Rank movement is descriptive and cannot by itself prove candidate quality. Proposed explanations will be tested for requirement coverage, evidence faithfulness, and unsupported-claim rate. A displayed claim is faithful only when its cited resume span and provenance directly support it.

D. Latency, Storage, API Usage and Cost

Future controlled runs will report extraction time, embedding time, lexical and vector retrieval time, fusion time, reconstruction time, reranking time, API usage, storage growth, and cost. Values will be distributions over repeated workloads rather than single convenient observations. The stored pilot execution time may verify completion, but is not used as a latency benchmark because network conditions, caching, corpus size, and hardware were not controlled.

VII. PRIVACY, FAIRNESS, SECURITY AND LIMITATIONS

Implemented controls include an S3 public-access block, server-side encryption, separate least-privilege AWS roles, environment-based service credentials, source and chunk provenance, deterministic identifiers, and avoidance of full resume text in logs. These are technical controls, not a complete governance programme. Authentication, end-user authorisation, retention enforcement, consent management, deletion workflows, and formal audit review remain to be integrated and tested.

The infrastructure template applies S3-managed encryption and restricts its Lambda roles to the required operations. AWS guidance distinguishes these storage protections from client-side data handling and key-governance decisions [23]. Similarly, Atlas Search and Vector Search protect retrieval functionality only when database credentials, network access, and collection privileges are configured correctly [13], [24]. Production deployment will require secret rotation, environment separation, and auditable access reviews beyond the code examined here.

Resumes contain direct identifiers and proxy attributes. Future evaluation should examine whether institution names, location, employment gaps, or writing style cause rank changes unsupported by job-relevant evidence [11]. Counterfactual checks must be paired with competence-preserving controls; invariance alone is not fairness. Recruiters must be able to correct extraction errors, inspect evidence, and override system order. No candidate should be rejected solely from a semantic relevance score.

The pilot has several limitations: one job description, a small candidate pool, no relevance labels, no extraction ground truth, and no controlled performance benchmark. BM25 settings are managed by Atlas Search, vector recall depends on index configuration, and section detection relies on recognisable headings. RRF can reward candidates with several retrieved chunks, and truncated reconstructed documents may omit late content. These limitations require further study.

VIII. RESULTS AND DISCUSSION

The retrieval output used 15 lexical chunk hits and 15 vector chunk hits. These represented eight unique candidates in each branch, with five shared candidates and an eleven-candidate union (Fig. 3). The result demonstrates complementarity for this query: three candidates were lexical-only and three were vector-only. It does not show that hybrid retrieval improves Recall@ K , since relevance labels are unavailable.

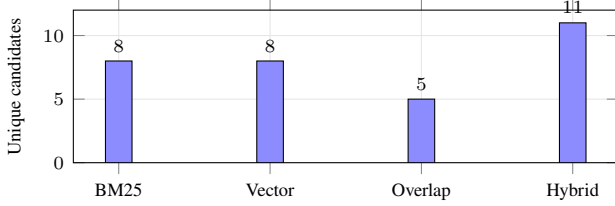


Figure 3. Unique-candidate coverage observed for the backend-engineer pilot query.

The stored reranking artefact received eleven candidates and returned five without reconstruction failures. Fig. 4 shows changes from hybrid ranks 1, 4, 10, 2, 9 to reranked positions 1, 2, 3, 4, 5. One vector-only candidate moved from tenth to third, while the initial leader remained first. This verifies reordering, not correctness; recruiter labels are required.

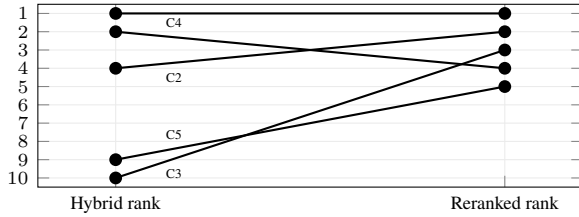


Figure 4. Retrieval-to-reranking rank movement for five anonymised candidates in the pilot output.

A. Planned Comparative Experiments

The first controlled experiment will compare BM25-only, vector-only, and fused retrieval under identical chunking, corpus, and retrieval depths. Recall@ K will be the primary candidate-generation measure, supplemented by MRR and nDCG where graded labels exist [19]. An ablation will then replace candidate-level multi-chunk accumulation with a best-chunk-only rule. This isolates whether repeated evidence across sections improves retrieval or merely rewards longer resumes.

The reranking experiment will hold the retrieved pool fixed and compare its order before and after cross-encoder evaluation. At least two independent recruiters will label a common subset using a written rubric. Inter-rater agreement will be reported before the system is compared with either judgement. Kendall’s τ and Spearman’s ρ will describe order association [20], [21], while top- K agreement will expose changes within the operational shortlist. Score thresholds will not be selected from the test set.

Extraction will be evaluated separately because downstream ranking cannot recover text that was never detected. Page-level annotations will distinguish missing lines, incorrect reading order, and wrong page provenance. Duplicate uploads will test

content-hash idempotency, and modified versions will verify stale-chunk replacement. These tests focus on system behavior rather than inferring document quality from successful API responses.

System experiments will repeat each workload after a warm-up period and report medians together with dispersion. S3 event delivery, Textract completion, embedding, retrieval, and reranking will be timed independently. AWS documents asynchronous completion through SNS [5], while S3 event notifications may be delivered more than once; idempotent processing is therefore an operational requirement rather than an optional optimisation [22].

IX. CONCLUSION AND FUTURE WORK

This work delivers an operational PDF-to-ranking pipeline in which asynchronous S3, Lambda, Textract, and SNS processing preserves extraction provenance from the first document event. Deterministic candidate chunks are embedded with voyage-4-large, indexed in MongoDB Atlas, retrieved through BM25 and vector search, fused through chunk-level RRF, and deduplicated into a focused candidate pool. Reconstructed resumes are then evaluated jointly with the job description by rerank-2.5, while retrieval ranks, matched chunks, and source references remain available for review.

The architecture addresses two recurring screening problems within one auditable workflow. BM25 retains exact technologies and role terminology, vector retrieval captures related meaning expressed with different wording, and RRF combines both rankings without forcing incompatible raw scores onto a common scale. Cross-encoder evaluation is concentrated on the retrieved pool, enabling richer job–resume comparison without applying pairwise processing to the entire corpus. The pilot demonstrates complementary candidate discovery and meaningful rank refinement while retaining a traceable connection to the indexed resume chunks.

Future implementation will extend this foundation with recruiter-labelled evaluation across multiple job families, requirement-to-resume evidence alignment, evidence-faithful explanations, and configurable human-review controls. Subsequent work will add alternative-opening matching and candidate communication based on the same reviewed evidence record, followed by systematic measurement of extraction quality, retrieval coverage, ranking agreement, latency, storage growth, and operational cost.

REFERENCES

- [1] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [3] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.
- [4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Recip-

- rocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd ACM SIGIR*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.
- [5] Amazon Web Services, “Calling Amazon Textract asynchronous operations,” *Amazon Textract Developer Guide*. [Online]. Available: <https://docs.aws.amazon.com/textract/latest/dg/api-async.html>. [Accessed: Oct. 1, 2026].
 - [6] E. Kesim and A. Deliahmetoglu, “Named entity recognition in resumes,” arXiv:2306.13062, 2023.
 - [7] M. Zhang, K. Jensen, S. Sonniks, and B. Plank, “SkillSpan: Hard and soft skill extraction from English job postings,” in *Proc. NAACL-HLT*, 2022, pp. 4962–4984.
 - [8] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
 - [9] M. T. Ribeiro, S. Singh, and C. Guestrin, ““Why should I trust you?”: Explaining the predictions of any classifier,” in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
 - [10] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
 - [11] D. F. Mujtaba and N. R. Mahapatra, “Fairness in AI-driven recruitment: Challenges, metrics, methods, and future directions,” arXiv:2405.19699, 2024.
 - [12] Voyage AI, “Text embeddings,” 2026. [Online]. Available: <https://docs.voyageai.com/docs/embeddings>. [Accessed: Oct. 1, 2026].
 - [13] MongoDB, “MongoDB Vector Search overview,” 2026. [Online]. Available: <https://www.mongodb.com/docs/vector-search/>. [Accessed: Oct. 1, 2026].
 - [14] MongoDB, “Return the score details: MongoDB Search,” 2026. [Online]. Available: <https://www.mongodb.com/docs/search/query/score/get-details/>. [Accessed: Oct. 1, 2026].
 - [15] Voyage AI, “Rerankers,” 2026. [Online]. Available: <https://docs.voyageai.com/docs/reranker>. [Accessed: Oct. 1, 2026].
 - [16] Government of India, *The Digital Personal Data Protection Act, 2023*. New Delhi, India: Ministry of Electronics and Information Technology, 2023.
 - [17] New York City Council, “Local Law 144 of 2021: Automated employment decision tools,” New York City Administrative Code, enforcement effective Jul. 2023.
 - [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
 - [19] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
 - [20] M. G. Kendall, “A new measure of rank correlation,” *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.
 - [21] C. Spearman, “The proof and measurement of association between two things,” *Amer. J. Psychol.*, vol. 15, no. 1, pp. 72–101, 1904.
 - [22] Amazon Web Services, “Amazon S3 event notifications,” *Amazon S3 User Guide*, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html>. [Accessed: Oct. 1, 2026].
 - [23] Amazon Web Services, “Protecting data with server-side encryption using Amazon S3 managed encryption keys,” *Amazon S3 User Guide*, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingServerSideEncryption.html>. [Accessed: Oct. 1, 2026].
 - [24] MongoDB, “MongoDB Search text operator,” 2026. [Online]. Available: <https://www.mongodb.com/docs/atlas/atlas-search/text/>. [Accessed: Oct. 1, 2026].
 - [25] Voyage AI, “Flexible dimensions and quantization,” 2026. [Online]. Available: <https://docs.voyageai.com/docs/flexible-dimensions-and-quantization>. [Accessed: Oct. 1, 2026].
 - [26] Voyage AI, “Reranker API reference,” 2026. [Online]. Available: <https://docs.voyageai.com/reference/reranker-api>. [Accessed: Oct. 1, 2026].